

L'infrastructure en code

CloudFormation : décrire, déployer, mettre à jour, détruire

CL-AWS2 — Jour 5 (jour 32 du cursus)

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer pourquoi **la console ne passe pas à l'échelle** — et pourquoi l'IaC est LE cœur du métier d'admin cloud moderne.
 - **Lire** un template CloudFormation : sections, **Ref**, **GetAtt**, paramètres, sorties.
 - **Écrire** un template simple (VPC + SG + instance) et le déployer en stack.
 - Utiliser les **changesets** pour prévoir l'impact d'une mise à jour, comprendre le **rollback** et la **détection de dérive**.
 - Situer **CodePipeline/CodeBuild** (CI/CD AWS) et **SAM** dans le paysage.
 - Savoir ce que **Terraform** (bloc 11) apportera en plus — et pourquoi on commence par CloudFormation.
-

Plan de la journée

1. **Pourquoi la console ne suffit plus** — le TP2 rejoué à la main, chronomètre en main.
2. **CloudFormation, les bases** — anatomie d'un template, lire avant d'écrire.
3. **Le cycle de vie d'une stack** — create, changesets, update, rollback, drift, delete.
4. **CI/CD AWS en panorama** — CodePipeline, CodeBuild, et SAM en une slide.

Au programme : 1 démo (mini StockLine en une commande), 2 exercices, 1 mini-TP non noté, quiz.

1. Pourquoi la console ne suffit plus

Souvenez-vous du TP2...

Pour monter l'architecture 3-tiers de StockLine, il vous a fallu créer, **dans le bon ordre** :

1 VPC, 4 sous-réseaux, 1 IGW, 1 NAT GW, 2 tables de routage et leurs associations, 3 security groups, 1 instance RDS, 1 launch template, 1 ASG, 1 ALB, 1 target group, 1 listener, des alarmes CloudWatch...

Soit **une trentaine de ressources**, des dizaines d'écrans de console, environ **4 heures** en suivant le guide — et combien d'erreurs de frappe, de SG mal référencés, d'AZ oubliées ?

Maintenant, imaginez la demande de votre chef :

« Très bien ! Refais-moi exactement la même chose pour l'environnement de **recette**, puis pour la **prod**. Et la semaine prochaine pour le client B. »

Ce qui ne va pas avec le « clic-clic »

- **Non répétable** : refaire = tout recommencer, avec de nouvelles erreurs. Deux environnements « identiques » ne le sont jamais.
- **Non documenté** : l'infrastructure n'existe que dans la console. Qui a ouvert ce port ? Pourquoi ? Aucune trace fiable.
- **Non testable** : impossible de faire relire « des clics » à un collègue avant de les faire en prod.
- **Non réversible** : supprimer proprement 30 ressources dans le bon ordre, de tête... vous avez vécu le teardown du TP2.
- **Dérive garantie** : chaque correctif manuel éloigne un peu plus la réalité de ce qu'on croit avoir.

La réponse du métier : **l'Infrastructure as Code (IaC)** — décrire l'infrastructure dans des **fichiers texte**, versionnés dans git, relus, rejoués.

C'est le premier mot de notre devise : **automatiser**, superviser, sécuriser.

L'IaC : l'infrastructure devient un fichier

Avec l'IaC, le fichier devient la **source de vérité** :

- **Versionnée** : `git log` raconte l'histoire de votre infrastructure (qui, quoi, quand, pourquoi).
- **Reliable** : une revue de code avant de toucher à la prod.
- **Répétable** : dev, recette, prod = le même fichier avec des paramètres différents.
- **Réversible** : détruire = une commande ; l'ordre est calculé pour vous.
- **Auto-documentée** : le fichier EST la documentation, toujours à jour.

Le paysage des outils :

Outil	Périmètre	Dans le cursus
CloudFormation	AWS uniquement, natif, gratuit	aujourd'hui
Terraform	multi-cloud (AWS, Azure...)	bloc 11 (7 jours)
SAM / CDK	surcouches AWS de CloudFormation	une slide chacun

Pourquoi commencer par CloudFormation ?

- **Zéro installation, zéro état à gérer** : c'est un service AWS, la console et la CLI suffisent — idéal pour apprendre les concepts.
- **Gratuit** : vous ne payez que les ressources créées, pas le service.
- Il est **partout** : SAM, CDK, Elastic Beanstalk, une bonne partie des solutions du marché... génèrent du CloudFormation. Savoir le **lire** est une compétence exigée en entreprise, même dans les équipes « full Terraform ».
- Les concepts (déclaratif, références entre ressources, plan avant application, rollback) sont **exactement ceux de Terraform** — le bloc 11 ira beaucoup plus vite grâce à aujourd'hui.

Et une conviction à installer dès maintenant : **on apprend à LIRE un template avant d'en écrire**. Comme pour une langue étrangère.

2. CloudFormation — les bases

Un template, une stack

Le principe en une image

template.yaml (fichier texte, dans git)	deploy ----->	STACK "abc-mini-stockline" +-----+
		VPC (créé)
		Subnet (créé)
Resources:		SG (créé)
MonVPC: ...	update	EC2 (créée)
MonSubnet: ...	----->	...
MonSG: ...		+-----+
MonInstance: ...	delete ----->	la stack regroupe TOUTES les ressources : créées, mises à jour et détruites ENSEMBLE, dans le bon ordre

Vocabulaire CloudFormation

Terme	Définition
Template	Le fichier YAML (ou JSON) qui décrit les ressources voulues
Stack	L' instanciation d'un template : le groupe de ressources réellement créées
Ressource	Un objet AWS déclaré dans le template (AWS::EC2::VPC ...)
Logical ID	Le nom de la ressource dans le template (MonVPC)
Physical ID	L'identifiant réel créé par AWS (vpc-0a1b2c ...)
Changeset	L' aperçu des changements avant une mise à jour
Rollback	Retour arrière automatique si une création/mise à jour échoue
Drift	Écart entre le template et la réalité (modifications manuelles)

Déjà vu ailleurs : template/stack = image/conteneur, AML/instance, task definition/task. **Le déclaratif, encore et toujours.**

Anatomie d'un template : les sections

```

AWSTemplateFormatVersion: "2010-09-09"      # toujours cette valeur
Description: Mini StockLine - VPC + EC2      # une phrase pour les humains

Parameters:                                  # les ENTREES : ce qui change d'un déploiement à l'autre
  Prefix:
    Type: String

Resources:                                   # le COEUR (seule section obligatoire) : les ressources
  MonVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 10.42.0.0/16

Outputs:                                    # les SORTIES : ce qu'on veut récupérer à la fin
  VpcId:
    Value: !Ref MonVPC

```

Trois questions pour lire n'importe quel template : **qu'est-ce qui entre** (Parameters) ? **qu'est-ce qui est créé** (Resources) ? **qu'est-ce qui sort** (Outputs) ?

Resources : Type + Properties

Chaque ressource déclare son **type** (quel objet AWS) et ses **propriétés** (sa configuration) :

```

Resources:
  WebSG:
    Type: AWS::EC2::SecurityGroup            # famille::service::objet
    Properties:
      GroupDescription: HTTP depuis Internet
      VpcId: !Ref MonVPC                    # reference a une autre ressource
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: 0.0.0.0/0

```

- Les types suivent le schéma **AWS::Service::Objet** — la **documentation de référence** (« Resource and property types ») liste chaque propriété : obligatoire ou non, type, valeurs.
- Vous n'avez **pas** à gérer l'ordre de création : CloudFormation le déduit des références (**WebSG** a besoin de **MonVPC** → le VPC sera créé d'abord).

Parameters : rendre le template réutilisable

```

Parameters:
  Prefix:

```

```

Type: String
Description: Votre trigramme (prefixe de nommage)
AllowedPattern: "[a-z]{3}"          # validation a la saisie
InstanceType:
  Type: String
  Default: t3.micro                  # valeur si non fournie
UbuntuAmiId:                       # l'AMI a jour, resolue par AWS
  Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
  Default:
/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id

```

- Un paramètre = **une valeur fournie au déploiement** : le même template sert pour dev, recette, prod (et pour chaque \$PREFIX de la salle !).
- `Default`, `AllowedValues`, `AllowedPattern`, `NoEcho` (pour les secrets) = garde-fous intégrés.
- Le type `AWS::SSM::Parameter::Value<...>` va chercher une valeur à jour dans Parameter Store — fini l'AMI codée en dur qui périmé.

Ref et GetAtt : le système nerveux du template

Les ressources se **réfèrent** entre elles — c'est ce qui remplace vos copier-coller d'IDs entre écrans de console :

```

VpcId: !Ref MonVPC                  # Ref -> l'ID "principal" (vpc-xxxx)
SubnetId: !Ref MonSubnetPublic      # Ref -> subnet-xxxx
GroupSet:
  - !GetAtt WebSG.GroupId           # GetAtt -> UN attribut precis
PublicUrl: !Sub "http://${MonInstance.PublicIp}:8000" # Sub -> interpolation

```

Fonction	Retourne	Exemple
<code>!Ref Ressource</code>	l'identifiant « par défaut » de la ressource	<code>vpc-0a1b...</code>
<code>!Ref Parametre</code>	la valeur du paramètre	<code>t3.micro</code>
<code>!GetAtt Res.Attr</code>	un attribut précis (IP publique, ARN, DNS...)	<code>15.236.4.18</code>
<code>!Sub "...\${X}..."</code>	une chaîne avec interpolation	URL construite

Ce que `Ref` et `GetAtt` retournent **dépend du type** : c'est écrit dans la doc de chaque ressource (section « Return values »).

Outputs : exposer ce qui compte

```

Outputs:
  StockLineUrl:
    Description: URL de l'API StockLine
    Value: !Sub "http://${MonInstance.PublicIp}:8000/sante"

```

```
VpcId:
  Description: ID du VPC (pour d'autres stacks)
  Value: !Ref MonVPC
  Export:                                # optionnel : partage inter-stacks
    Name: !Sub "${Prefix}-vpc-id"
```

- Les outputs s'affichent en console et en CLI :

```
aws cloudformation describe-stacks --stack-name "$PREFIX-mini-stockline" \
  --query "Stacks[0].Outputs" --output table
```

- **Export** permet à une **autre stack** d'importer la valeur (**!ImportValue**) — c'est ainsi qu'on découpe une grosse infra en stacks réseau / données / application.

Méthode : comment LIRE un template inconnu

Votre futur quotidien : on vous tend un template de 400 lignes. Méthode en 5 étapes :

1. **Description** : que prétend-il faire ?
2. **Parameters** : de quoi a-t-il besoin ? Y a-t-il des valeurs par défaut dangereuses (0.0.0.0/0 ?...) ?
3. **Resources** : listez les **types** (juste les types !) pour dessiner l'architecture de tête. **grep "Type:"** fait ça très bien.
4. Suivez les **Ref/GetAtt** pour comprendre le câblage (qui dépend de qui).
5. **Outputs** : qu'est-ce que la stack promet de fournir ?

```
grep -E "Type: AWS::" template-mini-stockline.yaml
```

Ne lisez jamais un template ligne à ligne de haut en bas : lisez-le **par section**, comme on lit un plan.



Exercice 5-1 — Lire un template avant de l'exécuter

Individuel, 30 minutes, sans déployer.

On vous remet `code/08-cl-aws2/template-mini-stockline.yaml` (celui de la démo de cet après-midi). Sans l'exécuter :

1. dessinez l'architecture qu'il décrit (ASCII ou papier) ;
2. répondez à 8 questions de lecture (que retourne ce **Ref** ? pourquoi ce **DependsOn** ? qui appelle qui ?) ;
3. relevez le point de sécurité discutable du template (il y en a un, assumé pour la formation).

Fichier : `exercices/08-cl-aws2/exercice-5-1-lire-un-template.md`

C'est l'exercice le plus « métier » de la journée : en entreprise, vous lirez 10 templates pour 1 que vous écrirez.

3. Le cycle de vie d'une stack

create → update (changeset) → rollback → delete

Créer une stack

```
# Valider la syntaxe AVANT (réflexe systématique)
aws cloudformation validate-template \
  --template-body file:///template-mini-stockline.yaml

# Créer la stack
aws cloudformation create-stack \
  --stack-name "$PREFIX-mini-stockline" \
  --template-body file:///template-mini-stockline.yaml \
  --parameters ParameterKey=Prefix,ParameterValue=$PREFIX \
  --region eu-west-3

# Suivre les événements (le "journal de bord" de la stack)
aws cloudformation describe-stack-events \
  --stack-name "$PREFIX-mini-stockline" \
  --query "StackEvents[].[Timestamp,LogicalResourceId,ResourceStatus]" \
  --output table
```

États à connaître : **CREATE_IN_PROGRESS** → **CREATE_COMPLETE** (ou **ROLLBACK_...** si échec). **validate-template** vérifie la **syntaxe**, pas la faisabilité : une AMI inexistante passera la validation et échouera à la création.

Le rollback : tout ou rien

Si **une seule ressource** échoue à la création, CloudFormation **détruit tout ce qu'il venait de créer** : c'est le rollback.

CREATE_IN_PROGRESS	MonVPC	✓
CREATE_IN_PROGRESS	MonSubnet	✓
CREATE_FAILED	MonInstance	X (ex. type d'instance invalide)
ROLLBACK_IN_PROGRESS		→ suppression du subnet, du VPC...
ROLLBACK_COMPLETE		→ stack vide, à supprimer puis recréer

- C'est une **protection** : pas d'infrastructure à moitié montée qui traîne (et qui coûte).
- Une stack en **ROLLBACK_COMPLETE** après un échec de **création** est inutilisable : **delete-stack**, corriger, recréer.
- **Le diagnostic est dans les events** : cherchez le premier **CREATE_FAILED**, colonne « status reason ». Toujours.

Vous provoquerez (volontairement) un rollback au mini-TP de cet après-midi.

Mettre à jour : les changesets

Jamais de mise à jour à l'aveugle. Le changeset montre ce qui VA se passer :

```
aws cloudformation create-change-set \  
  --stack-name "$PREFIX-mini-stockline" \  
  --change-set-name ajout-port-8000 \  
  --template-body file://template-mini-stockline.yaml \  
  --parameters ParameterKey=Prefix,UsePreviousValue=true  
  
aws cloudformation describe-change-set \  
  --stack-name "$PREFIX-mini-stockline" \  
  --change-set-name ajout-port-8000 \  
  --query "Changes[0].ResourceChange.[Action,LogicalResourceId,Replacement]" \  
  --output table
```

La colonne à surveiller : **Replacement**.

Valeur	Signification
False	modification en place, sans interruption
Conditional	remplacement possible selon la valeur
True	la ressource sera détruite et recréée (nouvelle IP, données perdues !)

Puis **execute-change-set** pour appliquer — ou supprimer le changeset si le plan ne convient pas. Vous retrouverez ce réflexe à l'identique avec **terraform plan** au bloc 11.

Supprimer — et protéger ce qui doit survivre

```
aws cloudformation delete-stack --stack-name "$PREFIX-mini-stockline"  
aws cloudformation wait stack-delete-complete \  
  --stack-name "$PREFIX-mini-stockline"
```

CloudFormation détruit **dans l'ordre inverse des dépendances** — le teardown de rêve comparé au TP2.

Deux subtilités d'exploitation :

- **DeletionPolicy: Retain | Snapshot** sur une ressource : à la suppression de la stack, la ressource est conservée (ou sauvegardée). Réflexe vital sur une base de données de prod.
- **DELETE_FAILED** : une ressource refuse de partir (bucket S3 non vide, SG encore référencé, ENI attachée...). On corrige la cause, ou on retente en excluant la ressource (**--retain-resources**) — puis on nettoie à la main. Le guide formateur du bloc détaille le cas.

Drift detection : quand la réalité s'éloigne du code

La **dérive** (drift) : quelqu'un modifie une ressource de la stack à la **main** (console, CLI). Le template dit une chose, la réalité une autre.

```
aws cloudformation detect-stack-drift --stack-name "$PREFIX-mini-stockline"
aws cloudformation describe-stack-resource-drifts \
  --stack-name "$PREFIX-mini-stockline" \
  --query "StackResourceDrifts[].[LogicalResourceId,StackResourceDriftStatus]" \
  --output table
```

Statuts : **IN_SYNC**, **MODIFIED** (propriétés changées), **DELETED** (ressource supprimée à la main).

Pourquoi c'est grave : la prochaine mise à jour de stack peut **écraser** le correctif manuel (fait un soir d'incident, jamais reporté dans le template)... ou échouer de façon incompréhensible.

Règle d'équipe : **toute modification passe par le template**. La console devient un outil de **lecture**.



Démo 5-1 — Un mini StockLine en une commande

Le formateur déroule, vous suivez — puis vous rejouerez tout au mini-TP.

1. Lecture rapide du template `template-mini-stockline.yaml` (méthode des 5 étapes).
2. `validate-template`, puis `create-stack` — pendant la création : lecture des events en direct, ordre des ressources.
3. Récupération de l'URL dans les **Outputs** → l'API StockLine répond (`/sante`, `/produits`).
4. Modification manuelle d'un SG en console → **drift detection** le détecte.
5. Changeset « type d'instance t3.micro → t3.small » → lecture du **Replacement**.
6. `delete-stack` : tout disparaît, dans le bon ordre, en une commande.

Fichiers : `demos/08-cl-aws2/demo-5-1-cloudformation.md`, `code/08-cl-aws2/template-mini-stockline.yaml`

Le TP2 vous avait pris 4 heures. Chronomètre en main : **environ 4 minutes**.

Écrire son premier template : la démarche

Pour écrire (et non plus lire), procédez **par accrétion** — jamais 200 lignes d'un coup :

1. **Squelette** : `AWSTemplateFormatVersion`, `Description`, une ressource simple (le VPC).
2. `validate-template` → `create-stack` → ça marche ? On continue.
3. Ajoutez une ressource à la fois (subnet, puis IGW, puis routes, puis SG, puis instance), en câblant avec `Ref/GetAtt`. Mise à jour par **changeset** à chaque étape.
4. Paramétrez à la fin (`Prefix`, `InstanceType`...) — d'abord faire marcher, ensuite généraliser.
5. Ajoutez les **Outputs** qui serviront (URL, IDs).

Outils d'aide : la **doc de référence** des types (toujours ouverte dans un onglet), `cfn-lint` (linter installable : `pip install cfn-lint`), et les exemples de la documentation AWS.

C'est l'objet de l'exercice 5-2 — sur un template **déjà écrit, mais cassé**.



Exercice 5-2 — Réparer un template cassé

En binômes, 45 minutes.

Le fichier `code/08-cl-aws2/template-casse.yaml` devrait déployer un VPC + SG + instance nginx. Il contient **3 erreurs** :

- une que `validate-template` attrape ;
- une qui échoue **au déploiement** (et déclenche un rollback — lisez les events !) ;
- une qui laisse la stack se créer... mais l'application **injoignable** (erreur de logique).

Trouvez-les, corrigez-les, prouvez que ça marche (le nginx répond), **détruisez la stack**.

Fichier : `exercices/08-cl-aws2/exercice-5-2-template-casse.md` Corrigé commenté : `code/08-cl-aws2/template-corrige.yaml` (après l'exercice !)



L'instance est une t3.micro : quelques centimes — mais teardown obligatoire.

Les limites de CloudFormation — et la suite

CloudFormation est excellent pour apprendre et très utilisé, mais :

- **AWS uniquement** : rien pour Azure (bloc 9 !), GCP, ou vos ressources annexes (DNS externe, GitHub...).
- **Verbeux** : YAML long, pas de vraie logique (boucles, conditions limitées).
- **Modularité limitée** : les nested stacks existent mais restent lourdes à l'usage.
- État géré par AWS : confortable... mais opaque quand ça coince (`DELETE_FAILED`, ressources orphelines).

Au **bloc 11 (CL-IAC, 7 jours)**, Terraform répondra point par point : **multi-cloud** (AWS + Azure avec le même outil), **state** explicite et partageable, **modules** réutilisables, **plan** systématique. Vous y recoderez toute l'infra du TP2.

Tout ce que vous avez appris aujourd'hui (déclaratif, références, plan avant application, dérive) **se transpose directement**. CloudFormation est votre langue d'apprentissage ; Terraform sera votre langue de travail.

4. CI/CD AWS — panorama

Et si plus personne ne lançait les commandes ?

CI/CD : définitions en 2 minutes

Vous avez maintenant : du code applicatif (StockLine), des images de conteneurs (J4), des templates d'infra (aujourd'hui). Tous vivent dans **git**. Qui exécute les commandes de déploiement ?

- **CI — intégration continue** : à chaque commit, une machine **vérifie** automatiquement (tests, lint, build de l'artefact — zip, image, template validé).

- **CD — déploiement continu** : si les vérifications passent, la même machine **déploie** automatiquement (ou après une approbation humaine).

Bénéfice pour l'admin cloud : plus de déploiement « depuis le poste de Kevin » — un processus **tracé, répétable, révocable**. Automatiser, toujours.

Vous pratiquerez le CI/CD pour de vrai au **TP4 (GitHub Actions + Terraform + Ansible)**. Aujourd'hui : savoir nommer et situer les briques AWS.

Les briques AWS : CodePipeline, CodeBuild, CodeDeploy

Service	Rôle	Équivalent marché
CodePipeline	le chef d'orchestre : enchaîne les étapes (source → build → deploy)	GitHub Actions (workflow), GitLab CI
CodeBuild	exécute une étape dans un conteneur éphémère (tests, build d'image, validation de template)	le « runner » / « job »
CodeDeploy	stratégies de déploiement fines (EC2, ECS, Lambda : rolling, blue/green, canary)	Argo Rollouts, Octopus

```
git push —> CodePipeline
           [Source: commit] -> [Build: CodeBuild      ] -> [Deploy:
CloudFormation /
                                tests + cfn-lint +      ECS update-service
/
                                docker build->push ECR ]   CodeDeploy]
                                (approbation manuelle possible avant
la prod)
```

Le marché utilise majoritairement GitHub Actions/GitLab CI pour l'orchestration — mais les cibles (**CloudFormation**, **ECR**, **ECS**) restent exactement celles vues cette semaine.

Démo conceptuelle — un pipeline sous les yeux

Pas de TP CI/CD aujourd'hui (le TP4 y est entièrement consacré). Le formateur vous fait visiter, en console, un pipeline pré-construit :

1. Le déclencheur : un commit sur la branche **main** du dépôt.
2. L'étape **Build** : les logs CodeBuild — on y reconnaît **cfn-lint** et **aws cloudformation validate-template...** vos commandes du matin !
3. L'étape **Deploy** : CodePipeline crée un **changeset** puis attend une **approbation manuelle** avant **execute-change-set**.

Ce qu'il faut retenir :

- Un pipeline n'invente rien : il **rejoue vos commandes**, à chaque commit, sans oubli ni fatigue.

- L'humain reste dans la boucle **là où on le décide** (l'approbation).
- IAM partout : le pipeline a un **rôle**, aux droits limités — pas les clés d'un admin.

SAM, en une slide

AWS SAM (Serverless Application Model) : une **surcouche de CloudFormation** dédiée au serverless que vous avez vu en J1-J3.

- Des types raccourcis : `AWS::Serverless::Function` remplace ~40 lignes (fonction Lambda + rôle + permissions + déclencheur API Gateway) par ~10.
- Une CLI dédiée : `sam build`, `sam deploy`, `sam local invoke` (tester la Lambda **sur son poste**).
- À la fin, `sam deploy...` génère et déploie un **template CloudFormation** : tout ce que vous avez appris aujourd'hui s'applique (stacks, changesets, rollback).

À retenir pour l'examen et les entretiens : « *SAM = CloudFormation spécialisé serverless* ». Si un jour vous industrialisez le TP3, c'est l'outil naturel.



Mini-TP — Une stack de bout en bout (non noté)

En autonomie, ~1 h 15, seul ou en binôme. Le grand final de la journée :

1. **Déployer** la stack fournie (VPC + SG + EC2 mini StockLine) avec votre `$PREFIX`.
2. **Mettre à jour** par changeset (nouveau port autorisé, puis type d'instance) — lire et interpréter le changeset AVANT d'exécuter.
3. **Provoquer un rollback** (mise à jour vers un type d'instance inexistant) et raconter, events à l'appui, ce qui s'est passé.
4. **Détecter une dérive** que vous aurez créée vous-même en console.
5. **Détruire**, et prouver qu'il ne reste rien.

Fichier : `tp/08-cl-aws2/tp-mini-cloudformation.md` — points de contrôle à chaque étape, dépannage inclus.



t3.micro + VPC sans NAT GW : quelques centimes pour l'après-midi. Teardown vérifié avant de partir.

Récap de la journée

- La console ne **passé pas à l'échelle** : non répétable, non documentée, non relisible. L'laC met l'infrastructure **dans git**.
- Un **template** (YAML) décrit ; une **stack** instancie. Sections : **Parameters** (entrées), **Resources** (le cœur), **Outputs** (sorties).
- **!Ref** = l'ID principal, **!GetAtt** = un attribut précis, **!Sub** = interpolation — le câblage qui remplace vos copier-coller.
- Cycle de vie : `validate` → `create` → **changeset** (lire **Replacement** !) → `execute` → `delete`. Échec = **rollback** automatique ; modification manuelle = **drift**.
- CI/CD AWS : **CodePipeline** orchestre, **CodeBuild** exécute — le pipeline rejoue vos commandes à chaque commit. **SAM** = CloudFormation du serverless.
- Terraform (bloc 11) généralisera tout ça : multi-cloud, state, modules.

Quiz de fin de journée

10 questions — répondez sur papier, correction demain matin

Quiz — questions 1 et 2

Question 1. Quelle est la différence entre un template et une stack ?

A. Aucune, ce sont deux synonymes B. Le template est le fichier de description ; la stack est le groupe de ressources créées à partir de lui C. La stack est le fichier YAML, le template est ce qui tourne D. Le template ne sert qu'à Terraform

Question 2. Quelle est la SEULE section obligatoire d'un template CloudFormation ?

A. **Parameters** B. **Outputs** C. **Resources** D. **Description**

Quiz — questions 3 et 4

Question 3. Dans un template, `!Ref MonVPC` retourne :

A. Toujours l'ARN de la ressource B. L'identifiant « par défaut » de la ressource (ici l'ID du VPC) C. Le nom logique en majuscules D. L'adresse IP du VPC

Question 4. Pour obtenir l'adresse IP publique d'une instance déclarée `MonInstance`, vous utilisez :

A. `!Ref MonInstance` B. `!GetAtt MonInstance.PublicIp` C. `!Sub MonInstance` D. `!ImportValue MonInstance`

Quiz — questions 5 et 6

Question 5. Pendant la création d'une stack, une ressource échoue. Que fait CloudFormation par défaut ?

A. Il ignore la ressource et continue B. Il met la stack en pause et attend une intervention C. Il déclenche un rollback : les ressources déjà créées sont supprimées D. Il facture les ressources en échec

Question 6. À quoi sert un `changeset` ?

A. À versionner le template dans git B. À visualiser les changements qu'une mise à jour appliquerait, AVANT de l'exécuter C. À changer de région sans redéployer D. À convertir un template JSON en YAML

Quiz — questions 7 et 8

Question 7. Dans un `changeset`, `Replacement: True` sur une ressource signifie :

A. La ressource sera modifiée sans interruption B. La ressource sera détruite puis recréée (nouvel ID, données locales perdues) C. La ressource sera dupliquée D. Le `changeset` a échoué

Question 8. Qu'est-ce que le « drift » d'une stack ?

A. La lenteur de création des ressources B. L'écart entre ce que décrit le template et l'état réel des ressources (modifications manuelles) C. Le déplacement d'une stack vers une autre région D. La suppression automatique des vieux templates

Quiz — questions 9 et 10

Question 9. Quel service AWS orchestre les étapes source → build → deploy d'un pipeline ?

A. CodeBuild B. CloudFormation C. CodePipeline D. Systems Manager

Question 10. Pourquoi apprendrons-nous Terraform au bloc 11 alors que CloudFormation existe ?

A. CloudFormation est payant, Terraform est gratuit B. Terraform est multi-cloud (AWS ET Azure), avec un état explicite et des modules réutilisables C. CloudFormation ne peut pas créer de VPC D. Terraform ne nécessite pas d'apprendre de syntaxe

Réponses expliquées demain matin — et dans le guide formateur.

À demain !

Jour 6 : bien architecturer, bien dépenser

Dernier jour du bloc AWS ! Au menu : les 6 piliers Well-Architected revisités avec VOS exemples, l'art de lire (et réduire) une facture AWS — la facture de **Negoce+** vous attend —, les 3 architectures de référence côte à côte, et le **grand quiz de synthèse** format Cloud Practitioner.

Ce soir : `aws cloudformation list-stacks` → toutes vos stacks en `DELETE_COMPLETE`.